50 Proven Prompts to Build a Full-Stack Web Application

Frontend → Backend → Database (plus Auth, Testing, and Deployment)

How to use this doc (fast):

1) Pick a stack (example: React + Node/Express + PostgreSQL).

2) Copy the prompts in order. Paste into your AI assistant (ChatGPT / Copilot / Claude / etc.).

3) Replace placeholders like {APP_NAME}, {FEATURES}, {DB_NAME}.

4) After each prompt, run the code, confirm it works, then move to the next prompt.

Project Spec Template (paste once before Prompt #1):

```
APP_NAME: {APP_NAME}
GOAL: {1 sentence goal}
USERS: {user types}
CORE FEATURES: {feature list}
STACK: {frontend}, {backend}, {database}
AUTH: {email/password | OAuth | none}
DEPLOY TARGET: {Vercel/Netlify + Render/Fly/Azure/etc.}
NON-FUNCTIONAL: {performance, security, logging, monitoring}
STYLE: Clean UI, mobile-first, accessible, simple.
```

The 50 Prompts (copy-paste)

Tip: paste prompts one by one. After each prompt, run the code, commit, and move to the next prompt.

Prompt #1: Define scope & MVP

```
Act as a senior product engineer. Based on the Project Spec, produce:
- MVP scope (must-have vs nice-to-have)
- Key user flows (happy path + edge cases)
- Data objects (entities) and relationships
Keep it short and actionable.
```

Prompt #2: System architecture map

```
Create an architecture plan for this full-stack app:
- Frontend pages/components
- Backend modules/routes
- Database tables + indexes
- Auth approach
- Error handling and logging
Return as a bullet list + a simple diagram in text.
```

Prompt #3: API contract first

```
Design the API contract (REST):
- Endpoints, methods, request/response JSON
- Validation rules
- Error shapes (standard format)
- Pagination and filtering if needed
Return a concise OpenAPI-style outline.
```

Prompt #4: DB schema draft

```
Design the database schema:
- Table definitions (columns, types, constraints)
- Relationships and cascade rules
- Indexes that matter
- Seed data plan
Return SQL (PostgreSQL).
```

Prompt #5: Folder structure

```
Propose a repository structure (monorepo or separate):
- frontend/
- backend/
- shared/ (types)
Include scripts for dev, test, and build.
```

Prompt #6: Local dev checklist

```
Write a step-by-step local setup checklist (fresh machine):
- prerequisites
- env vars
- run DB
- run backend
- run frontend
Also include common fixes when ports or env vars break.
```

Prompt #7: Frontend bootstrap

```
Generate the frontend starter (React + Vite + TypeScript):
- routing
- global layout
- API client wrapper (fetch/axios)
- environment config
Return exact commands + file contents.
```

Prompt #8: Design system basics

```
Create a minimal UI system:
- spacing scale
- typography scale
- button/input/card styles
- reusable components (Button, Input, Modal, Toast)
Ensure accessibility (labels, focus).
```

Prompt #9: Page list & wireframes

```
List the pages required for the MVP and provide quick wireframes in text:
- header/footer/nav
- empty/loading/error states per page.
```

Prompt #10: State management plan

```
Recommend state handling (React Query + context, or similar):
- what goes in server state vs UI state
```

```
- caching rules
- loading/error UX patterns.
```

Prompt #11: Auth screens

```
Generate UI screens for auth:
- sign up
- sign in
- forgot password (if used)
Include form validation and inline error messages.
```

Prompt #12: Dashboard / main screen UI

```
Generate the main dashboard UI for the core flow:
- data table or cards
- filters/search
- create/edit dialogs
Include responsive behavior.
```

Prompt #13: Detail view + edit flow

```
Add a detail page with edit mode:
- view state
- edit state with validation
- save/cancel UX
Use optimistic updates if appropriate.
```

Prompt #14: Reusable form builder

```
Create a reusable form helper:
- schema validation (Zod/Yup)
- reusable FormField component
- consistent error display
Show example usage on one form.
```

Prompt #15: Client-side routing guards

```
Add protected routes:
- redirect unauthenticated users to login
- preserve intended destination
- handle token expiry gracefully.
```

Prompt #16: Frontend error boundaries

```
Add an ErrorBoundary and a consistent error page:
- user-friendly messaging
- retry button
- log details in dev only.
```

Prompt #17: Frontend testing plan

```
Write a testing plan for frontend:
- unit tests for components
- integration tests for main flows
- recommended tools + sample test.
```

Prompt #18: Performance basics

```
Improve frontend performance:
- code splitting
- image optimization
- avoid re-renders
Provide concrete code changes.
```

Prompt #19: Backend bootstrap

```
Generate backend starter (Node.js + Express + TypeScript):
- linting, formatting
- env handling
- health endpoint
- structured logging
Return commands + file tree + key files.
```

Prompt #20: DB connection layer

```
Implement DB layer (Prisma/Knex/pg):
- connection setup
- migrations
- repository pattern
Include sample query functions.
```

Prompt #21: Auth implementation

```
Implement auth for the chosen approach:
- password hashing + salting (if email/password)
- JWT/session handling
- refresh strategy (if used)
- secure cookies (if web)
Include middleware + example routes.
```

Prompt #22: User model + validation

```
Create user entity:
- create user
- login
- get current user (/me)
Add validation + consistent error responses.
```

Prompt #23: CRUD: create

```
Implement the Create endpoint for the main entity:
- validation
- authorization rules
- DB write
- return JSON that matches the API contract.
```

Prompt #24: CRUD: read list

```
Implement List endpoint:
- pagination
- filtering/search
- sorting
- return total count
Include SQL/index notes.
```

Prompt #25: CRUD: read single

```
Implement Get-by-id endpoint:
- 404 handling
- authorization
- include related data if needed.
```

Prompt #26: CRUD: update

```
Implement update endpoint:
- partial update (PATCH) vs full update (PUT)
- validation rules
- concurrency strategy (updated_at or version)
```

Prompt #27: CRUD: delete

```
Implement Delete endpoint:
- soft delete vs hard delete
- audit fields
- authorization
Return response standard.
```

Prompt #28: Rate limiting + security headers

```
Add basic protection:
- rate limiting on auth routes
- security headers (helmet)
- CORS config for the frontend origin
Explain the settings.
```

Prompt #29: Central error handler

```
Implement a global error handler:
- map known errors to status codes
- preserve stack traces in dev only
- unify error JSON shape.
```

Prompt #30: Request logging + tracing

```
Add request logging:
- request id
- timing
- user id (if available)
Show example log line format.
```

Prompt #31: Backend tests

```
Create backend tests:
- unit tests for services
- integration tests for routes using a test DB
Provide at least 2 sample tests.
```

Prompt #32: API docs

```
Generate API documentation:
- OpenAPI spec file
- how to render Swagger UI
- examples for each endpoint.
```

Prompt #33: Background jobs (optional)

```
If this app needs async work (emails, processing):
- propose a job queue
- implement one sample job + worker
If not needed, say 'skip' and why.
```

Prompt #34: File uploads (optional)

```
If the app needs uploads:
- choose storage (S3/Azure Blob/local for dev)
- implement signed upload flow
- validate type/size
If not needed, say 'skip'.
```

Prompt #35: Migrations workflow

```
Set up migrations:
- create migration
- apply migration
- rollback strategy
Include scripts and best practices.
```

Prompt #36: Seed data

```
Create seed scripts for local/dev:
- sample users
- sample entities
- deterministic data
Include how to run it.
```

Prompt #37: Indexes & query tuning

```
Review the schema and propose indexes:
- explain why each index helps
- show the SQL
Also mention common slow queries to watch.
```

Prompt #38: Data validation at DB level

```
Add DB constraints:
- unique constraints
- foreign keys
- check constraints
Explain how they prevent bugs.
```

Prompt #39: Audit fields

```
Add audit columns:
- created_at, updated_at, deleted_at (if soft delete)
- created_by, updated_by (if needed)
Update code to write these fields.
```

Prompt #40: Transactions

```
Find one risky multi-step operation and wrap it in a transaction:
- show the code
- show rollback behavior on failure.
```

Prompt #41: Backups & restore plan

```
Create a simple backup/restore plan:
- dev
- staging
- production
Include retention + how to test restores.
```

Prompt #42: Observability hooks

```
Add basic observability:
- metrics endpoint (or logs-based metrics)
- structured logs for key actions
- health checks (liveness/readiness)
```

Prompt #43: Security review checklist

```
Review security:
- auth/session handling
- input validation
- injection risks
- secrets management
Return a checklist + quick fixes.
```

Prompt #44: Smoke test script

```
Write a smoke test script that hits:
- /health
- auth flow (if used)
- one create/read/update path
Return a runnable script (bash or node).
```

Prompt #45: Containerize backend

```
Create a Dockerfile for backend:
- small image
- non-root user
- env var usage
- healthcheck
Also write a docker-compose for local full stack.
```

Prompt #46: Deploy plan

```
Create a deploy plan for:
- frontend hosting
- backend hosting
- database hosting
Include env vars, secrets, and domain/HTTPS steps.
```

Prompt #47: CI pipeline

```
Create a CI pipeline that runs:
- lint
- tests
- build
- security checks (basic)
Return a GitHub Actions YAML (or Azure DevOps YAML if I request).
```

Prompt #48: Release checklist

```
Write a release checklist:
- migrations
- config checks
- monitoring/alerts
- rollback steps
Make it short and realistic.
```

Prompt #49: Prompt to iterate features safely

```
When I ask for a new feature, follow this workflow:
1) restate requirements + edge cases
2) propose minimal code changes
3) update API + DB if needed
4) update frontend
5) add tests
6) show how to verify locally
Return as a reusable instruction prompt.
```

Prompt #50: Prompt to debug production issues

```
When I report a bug, ask for:
- exact error message/logs
- steps to reproduce
- env details
Then propose:
- most likely root causes
- step-by-step investigation
- safest fix + verification
Return as a reusable instruction prompt.
```

One-pass Build Flow (suggested order)

1) Prompts 1–6: scope, architecture, API contract, schema, repo layout

2) Prompts 7–18: frontend foundation and UI flows

3) Prompts 19–34: backend + APIs + auth + tests

4) Prompts 35–44: DB reliability, observability, security

5) Prompts 45–50: containerize, deploy, CI, iterate

CTA (optional):

Want the full prompt pack? Comment “PROMPTS” and I’ll share it.